SQL SERVER / T-SQL

Fundamentos de programação de banco de dados

SP-STORED PROCEDURES

Prof. Me. Francisco Martins

STORED PROCEDURES DEFINIÇÃO

- Lotes (batches) de declarações SQL executadas como uma subrotina.
- Permitem centralizar a lógica de acesso aos dados em um único local, facilitando a manutenção e otimização de código.
- Usadas para validação de dados, controle de acesso, execução de declarações SQL complexas e outras situações.
- É possível ajustar permissões de acesso aos usuários definindo quem pode ou não executá-las.

STORED PROCEDURES - SINTAXE

- **Criar um procedimento armazenado:**

```
CREATE PROCEDURE sp_ nome do procedimento
(
    @parâmetro Tipo_dados, ... (Dados passados ao procedimento)
)
AS
[BEGIN]
    Bloco de código
[END]
GO
```

STORED PROCEDURES - EXECUÇÃO

- **Executar o SP**

- Para executar um Procedimento Armazenado, use a palavra-chave EXECUTE ou EXEC [são opcionais]

[EXEC | EXECUTE] nome_procedimento parâmetros

EX:

EXEC sp_teste @NomeAutor = 'Autor Teste';

STORED PROCEDURES

- **Criação e Execução de um sp** (Este procedimento imprime uma frase na tela).

```
CREATE PROCEDURE sp_teste
AS
SELECT 'Análise e Desenvolvimento de Sistemas - ADS' AS curso;

EXEC sp_teste; --EXECUTE
```

- O comando EXEC invoca ou executa o Procedimento sp_teste

STORED PROCEDURES

- **Criação e Execução de um sp** (Este procedimento apresenta o resultado do SELECT executado por ele que lista os nomes e preços dos livros). A vantagem é poder executar todo o código de forma rápida.

```
CREATE PROCEDURE sp_LivroValor
AS
BEGIN
   SELECT Nomelivro Título, PrecoLivro Valor
   FROM Livro;
END
GO

EXEC sp_LivroValor;
```

STORED PROCEDURES

- **Alteração de um sp** (Este procedimento altera o anterior inserindo a cláusula ou condição **WHERE**). Neste caso lista apenas os livros com preços iguais ou maiores que R$ 150,00.

```
ALTER PROCEDURE sp_LivroValor
AS
BEGIN
   SELECT Nomelivro Título, PrecoLivro Valor
   FROM Livro
   WHERE PrecoLivro >= 150.00;
END
GO
```

STORED PROCEDURES

- **Conhecer ou verificar o código de um sp** (Este comando lista o conteúdo (código) existente, seja interno do sistema ou criado pelo programador).
- A leitura do código de todos os sp_helptext permite conhecer outras formas de uso da linguagem SQL.

```
EXEC sp_helptext sp_LivroValor;

EXEC sp_helptext sp_helptext; --Código de todo o helptext
```

Resultados | Mensagens

	Text
1	CREATE PROCEDURE sp_LivroValor (
2	@ID as smallint) AS
3	BEGIN
4	SELECT NomeLivro Título, PrecoLivro Valor
5	FROM Livro
6	WHERE IdLivro = @ID;
7	END

STORED PROCEDURES

- **Criar um sp com código criptografado** (O comando WITH ENCRYPTION criptografa o código do sp).

```
CREATE PROCEDURE sp_LivroISBN
WITH ENCRYPTION
AS
BEGIN
    SELECT Nomelivro, ISBN13
    FROM Livro;
END
GO
```

Resultados | Mensagens

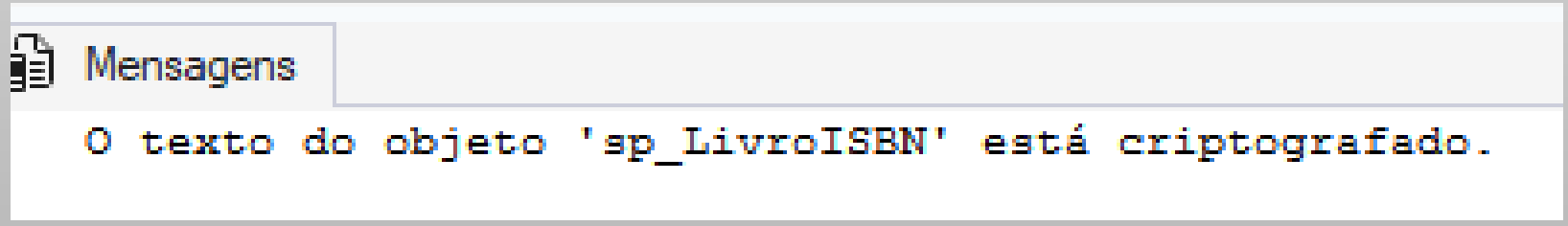
	Nomelivro	ISBN13
1	A Arte da Eletrônica	9788582604342
2	Vinte Mil Léguas Submarinas	9788582850022
3	O Investidor Inteligente	9788595080805
4	2001 - Uma Odisséia no Espaço	9788576571551
5	Fahrenheit 451	9788525052247
6	Admirável mundo novo	9788525056009

STORED PROCEDURES

- **Criar um sp com código criptografado** (O comando WITH ENCRYPTION criptografa o código do Procedimento).

```
CREATE PROCEDURE sp_LivroISBN
WITH ENCRYPTION
AS
BEGIN
   SELECT Nomelivro, ISBN13
   FROM Livro;
END
GO
```

```
EXEC sp_helptext sp_LivroISBN;
```



STORED PROCEDURES

- **sp_rename 'Nome Atual', 'Nome Novo';**

- **RENOMEAR** UM SP É DESACONSELHADO PELA MICROSOFT, PORQUE O NOME DO OBJETO NÃO É ALTERADO NO CATÁLOGO INTERNO DO SISTEMA OU MECANISMO DO BANCO DE DADOS.
- É RECOMDADO EXCLUIR E CRIAR NOVAMENTE.

STORED PROCEDURES

- **PARÂMETROS DE ENTRADA: A função CONCAT concatena o valor passado na execução do sp à palavra "Valor" impressa pelo sp, neste caso, o valor 63.**
- ***O comando "CREATE OR ALTER":** Cria ou altera um sp se ele já existir.

```
Create or ALTER PROCEDURE sp_teste1a (@p1 as int)
AS
SELECT CONCAT('Valor: ', @p1) AS Valor;

EXEC sp_teste1a 63;
```

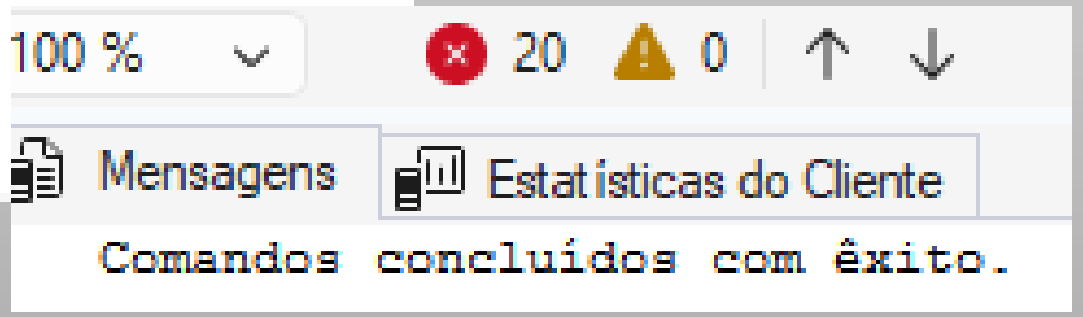
Resultados | Mensagens

	Valor
1	Valor: 63

STORED PROCEDURES

- **PARÂMETROS DE ENTRADA:** Temos que ter o cuidado de atribuir o mesmo tipo de dado da coluna da tabela ao parâmetro para não haver erro na comparação como **(WHERE IdLivro = @ID).**
-

```
ALTER PROCEDURE sp_LivroValor (
@ID as smallint) AS
BEGIN
    SELECT NomeLivro Título, PrecoLivro Valor
    FROM Livro
    WHERE IdLivro = @ID;
END
GO
```



STORED PROCEDURES

- **PARÂMETROS DE ENTRADA:** Temos que ter o cuidado de atribuir o mesmo tipo de dado da coluna da tabela ao parâmetro para não haver erro na comparação como **(WHERE IdLivro = @ID).**
-

```
ALTER PROCEDURE sp_LivroValor (
@ID as smallint) AS
BEGIN
    SELECT NomeLlivro Título, PrecoLivro Valor
    FROM Livro
    WHERE IdLivro = @ID;
END
GO
                    EXEC sp_LivroValor 104;
```

Resultados | Mensagens

	Título	Valor
1	Fahrenheit 451	43,56

STORED PROCEDURES

- **Passando 2 PARÂMETROS DE ENTRADA:** Temos que ter o cuidado de atribuir o mesmo tipo de dado das colunas da tabela aos parâmetros para que funcione corretamente**. Se inverter os parâmetros gera ERRO.**

```
CREATE OR ALTER PROCEDURE sp_teste (@p1 as int, @p2 AS VARCHAR(20))
AS
SELECT @p1 AS 'Parâmetro 01: ', @p2 AS 'Parâmetro 02: ';

EXEC sp_teste 63, 'Meu teste';
```

- **Se for usar invertido, indique o parâmetro**

```
EXEC sp_teste @p2 = 'Meu teste', @p1=63;
```

Resultados | Mensagens

	Parâmetro 01:	Parâmetro 02:
1	63	Meu teste

STORED PROCEDURES

- **Passando 2 PARÂMETROS DE ENTRADA:** Para passar dois parâmetros para o sp realizar um cálculo usando dados da tabela. Cálculo para saber quanto custaria comprar 5 livros ao preço atual.

```
ALTER PROCEDURE sp_LivroValor (
@ID as smallint, @Qtde SMALLINT)
AS
BEGIN
    SELECT NomeLivro Título, @Qtde Quantidade,
    PrecoLivro * @Qtde 'Valor Total'
    FROM Livro
    WHERE IdLivro = @ID;
END
```

```
EXEC sp_LivroValor 104, 5;
```

Resultados | Mensagens

	Título	Quantidade	Valor Total
1	Fahrenheit 451	5	217,80

STORED PROCEDURES

- **Sp para cadastrar dados:** Permite cadastrar dados de Editoras no banco de dados

```
CREATe OR ALTER PROCEDURE sp_cadastra_editora (
    @nome varchar(50))
AS
BEGIN
    INSERT INTO Editora(NomeEditora)
    VALUES (@Nome);
END
```

```
EXEC sp_cadastra_editora 'Minha Editora';
```

```
EXEC sp_cadastra_editora 'Editora UNIP-ADS';
```

Resultados | Mensagens

	IdEditora	NomeEditora
24	24	Minha Editora
25	25	Editora UNIP-ADS

STORED PROCEDURES

- **Sp para cadastrar dados:** Para não exibir a mensagem de "Linhas afetadas" inclui-se o comando **SET NOCOUNT ON**.

```
CREATe OR ALTER PROCEDURE sp_cadastra_editora (
    @nome varchar(50))
AS
BEGIN
    SET NOCOUNT ON
    INSERT INTO Editora(NomeEditora)
    VALUES (@Nome);
END
```

```
EXEC sp_cadastra_editora 'Minha Editora';
```

```
EXEC sp_cadastra_editora 'Editora UNIP-ADS';
```

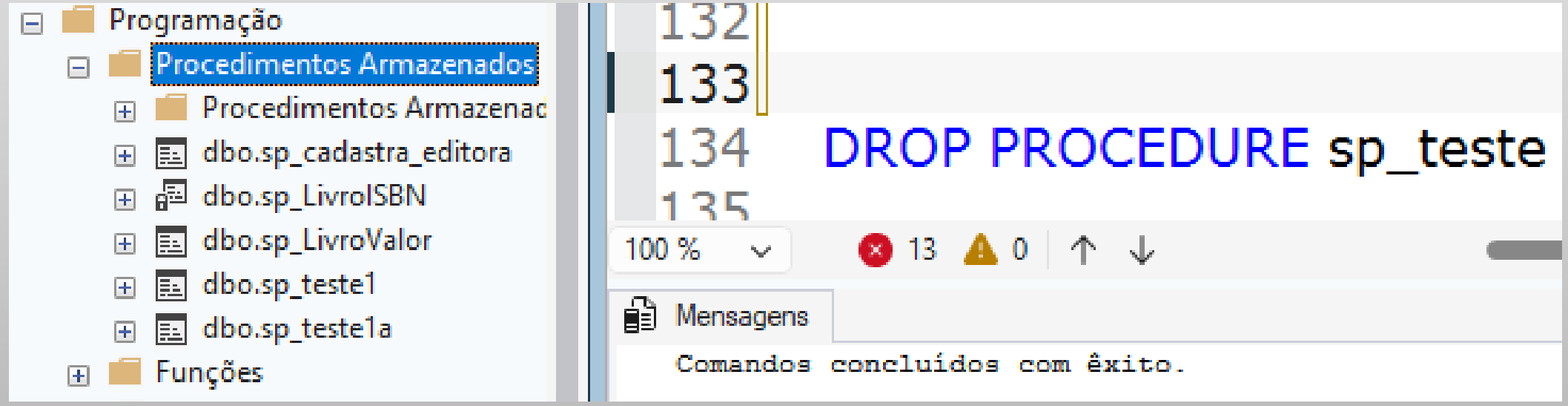
Resultados | Mensagens

	IdEditora	NomeEditora
24	24	Minha Editora
25	25	Editora UNIP-ADS

STORED PROCEDURES

- **Para excluir um sp utilizamos o comando DROP PROCEDURE** nome_procedimento.

```
DROP PROCEDURE sp_teste
```



SQL SERVER / T-SQL

Fundamentos de programação de banco de dados

TRIGGERS (Gatilhos)

Prof. Me. Francisco Martins

TRIGGERS (GATILHOS)

- Um Trigger (Gatilho) é um tipo especial de procedimento armazenado executado automaticamente quando um usuário realiza uma operação de modificação de dados em uma **Tabela** ou **View** especificada.

- As operações que podem disparar um trigger são:

  - **INSERT**
  - **UPDATE**
  - **DELETE**

TRIGGERS (GATILHOS)

- **Tipos de Triggers**

- No SQL Server existem três tipos principais de triggers:

  - DML Triggers (Data Manipulation Language): Disparados por eventos DML como INSERT, UPDATE, ou DELETE.
  - DDL Triggers (Data Manipulation Language): Disparados por eventos DDL como CREATE, ALTER ou DROP.
  - Logon Triggers: Disparados em resposta a eventos de logon do servidor

TRIGGERS (GATILHOS)

- **Modos de disparo de um Triggers**

- Um Trigger no SQL Server pode ser disparado de dois modos diferentes:

  - AFTER – **(Após)** O código presente no trigger é executado após todas as ações terem sido completadas na tabela especificada.
  - INSTEAD OF – **(Em vez de**) O código presente no trigger é executado no lugar da operação que causou o seu disparo

TRIGGERS (GATILHOS) - FLUXO DE TRANSAÇÕES

- 1- Verificação de IDENTITY INSERT.
- 2- Restrição (Constraint) de Nulos (NULL).
- 3- Checagem de tipos de dados.
- 4- Execução de trigger INSTEAD OF (a execução do DML pára aqui; esse trigger não é recursivo).
- **Apartir daqui são transações do trigger AFTER**
- 5-Restrição de PK (chave primária).
- 6- Restrição de "CHECK".
- 7- Restrição de FK (chave estrangeira).
- 8- Execução do DML e atualização do log de transações.
- 9- Execução do trigger AFTER
- 10- Commit da Transação (Confirmação).

TRIGGERS (GATILHOS)

- **Sintaxe Geral de um Trigger**

```
CREATE [OR ALTER] TRIGGER nome_trigger
ON tabela | view
[WITH ENCRYPTION]
    AFTER | INSTEAD OF [INSERT, UPDATE, DELETE]
AS
BEGIN
    Código do Trigger
END
GO
```

TRIGGERS (GATILHOS)

- **Criação de um Trigger AFTER INSERT**

```
CREATE OR ALTER TRIGGER tg_editora_cadastrada
ON Editora
AFTER INSERT
AS
BEGIN
    SELECT CONCAT('Há ', COUNT(*), ' editoras cadsatradas') AS cadastros
    FROM Editora;
END
GO
```

- **Este Trigger foi criado na tabela Editora, e será disparado / executado quando houver um cadastro de uma nova editora.**

TRIGGERS (GATILHOS)

- **Inserção de um novo registro na tabela Editora faz com que o Trigger seja disparado executando o código interno dele "SELECT"**

```
INSERT INTO  Editora Values ('Nova Editora trg');

SELECT * FROM Editora;
```

Resultados | Mensagens

	cadastros
1	Há 27 editoras cadsatradas

Resultados | Mensagens

	IdEditora	NomeEditora
26	26	Nova editora trg
27	27	Nova Editora trg

- **O primeiro resultado foi gerado pelo INSERT, o segundo pelo SELECT.**

TRIGGERS (GATILHOS)

- **Criação de um Trigger INSTEAD OF para bloquear o cadastro de autores na tabela.**

```
CREATE OR ALTER TRIGGER tg_bloqueia_autor
ON Autor
INSTEAD OF INSERT
AS
BEGIN
   PRINT 'Cadastro de autores está bloqueado';
END
GO
```

- **Este Trigger foi criado na tabela Autor, e será disparado / executado quando houver a tentativa de cadastro de um Autor.**

TRIGGERS (GATILHOS)

- **Inserção de um novo registro na tabela faz com que o Trigger seja disparado executando o código interno dele "PRINT"**

```
INSERT INTO  Autor Values ('José', 'de Alencar');
```

Mensagens

```
Cadastro de autores está bloqueado

(1 linha afetada)
```

- **O resultado foi gerado pelo comando INSERT INTO na tabela Autor.**

TRIGGERS (GATILHOS)

```
--Se precisar cadastrar uma autor, é preciso destivar o trigger

ALTER TABLE Autor
DISABLE TRIGGER tg_bloqueia_autor;

--Para habilitar novamente o trigger

ALTER TABLE Autor
ENABLE TRIGGER tg_bloqueia_autor;
```

TRIGGERS (GATILHOS)

```
--Para EXCLUIR um trigger
DROP TRIGGER tg_bloqueia_autor;

--Para visualizar os triggers
EXEC sp_helptrigger @tabname=Editora;
EXEC sp_helptrigger @tabname=Autor;

--Para visualizar todos os triggers do BD
SELECT * FROM sys.triggers
WHERE is_disabled = 0 OR IS_DISABLED = 1;
```

TRIGGERS (GATILHOS)

```
--Para visualizar os triggers de uma tabela específica
EXEC sp_helptrigger @tabname=Editora;
EXEC sp_helptrigger @tabname=Autor;
```

	trigger_name	trigger_owner	isupdate	isdelete	isinsert	isafter	isinsteadof	trigger_schema
1	tg_editora_cadastrada	dbo	0	0	1	1	0	dbo

	trigger_name	trigger_owner	isupdate	isdelete	isinsert	isafter	isinsteadof	trigger_schema
1	tg_insere_autor	dbo	0	0	1	1	0	dbo
2	tg_bloqueia_autor	dbo	0	0	1	0	1	dbo

TRIGGERS (GATILHOS)

```
--Para visualizar todos os triggers do BD
SELECT * FROM sys.triggers
WHERE is_disabled = 0 OR IS_DISABLED = 1;
```

- **O valor 0 indica os triggers desabilitados,**
- **O valor 1 indica os triggers habilitados**

	name	object_id	parent_class	parent_class_desc	parent_id	type	type_desc	create_date	modify_date
1	tg_editora_cadastrada	978102525	1	OBJECT_OR_COLUMN	1557580587	TR	SQL_TRIGGER	2026-10-13 19:02:26.260	2026-10-13 19:02:26.260
2	tg_insere_autor	1026102696	1	OBJECT_OR_COLUMN	1525580473	TR	SQL_TRIGGER	2026-10-13 19:32:33.300	2026-10-13 19:32:33.300
3	tg_bloqueia_autor	1298103665	1	OBJECT_OR_COLUMN	1525580473	TR	SQL_TRIGGER	2026-10-14 09:12:03.033	2026-10-14 09:12:03.033

TRIGGERS (GATILHOS)

```
--Um procedimento poderá ser executado direto ou
--por um trigger
CREATE OR ALTER PROCEDURE sp_consulta_autores
AS
BEGIN
    SELECT NomeAutor, SobrenomeAutor
    FROM Autor
    ORDER BY NomeAutor;
END
GO

EXEC sp_consulta_autores;
```

100 % 20 0

Resultados Mensagens

	NomeAutor	SobrenomeAutor
1	Alan	Turing
2	Aldous	Huxley
3	Arthur	Conan Doyle
4	Arthur	C. Clarke
5	Benjamin	Graham
6	Blaise	Pascal
7	Charles	Darwin
8	Christine	Bresnahan

TRIGGERS (GATILHOS)

```
--Este trigger executa o sp_consulta-autores
CREATE OR ALTER TRIGGER tg_insere_autor
ON Autor
AFTER INSERT
AS
BEGIN
    EXEC sp_consulta_autores;
END
GO
--O comando INSERT INTO dispara o trigger específico
INSERT INTO  Autor Values ('Addam', 'Smith');
```

100 % 26 0

Resultados Mensagens

	IdAutor	NomeAutor	SobrenomeAutor
1	48	Addam	Smith
2	40	Alan	Turing
3	32	Aldous	Huxley

100 % 26 0

Resultados Mensagens

	NomeAutor	SobrenomeAutor
1	Addam	Smith
2	Alan	Turing
3	Aldous	Huxley
4	Arthur	Conan Doyle
5	Arthur	C. Clarke

TRIGGERS (GATILHOS)

```
INSERT INTO  Autor Values ('Clarice', 'Lispector');

--Um Trigger que bloqueia uma ação necessária precisa ser
--desboqueado, neste caso o Trigger tg_bloqueia_autor
ALTER TABLE Autor
DISABLE TRIGGER tg_bloqueia_autor;
```

Resultados | Mensagens | Estatísticas

	NomeAutor	SobrenomeAutor
8	Christine	Bresnahan
9	Clarice	Lispector
10	Daniel	Barret

SQL SERVER / T-SQL

Fundamentos de programação de banco de dados

FUNÇÕES UDF

Prof. Me. Francisco Martins

FUNÇÕES UDF